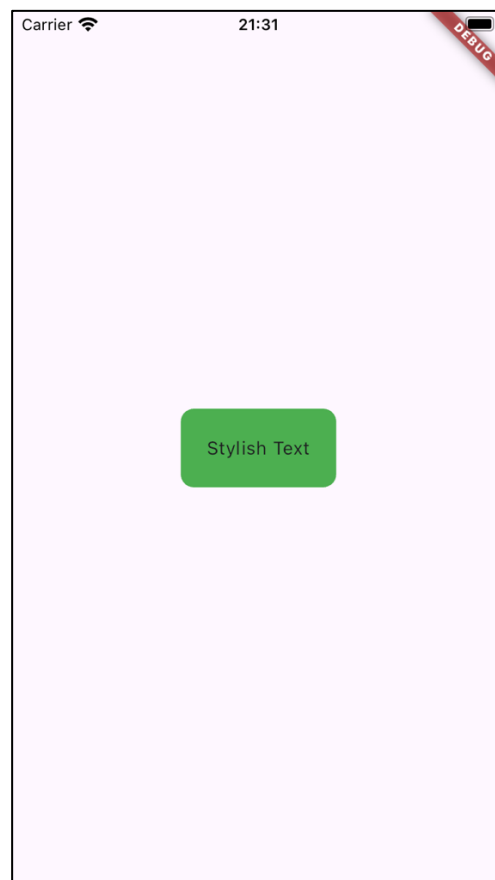


- 例として Text ウィジェットが Container ウィジェットの中にネストされる：

```
Container(  
  padding: EdgeInsets.all(20),  
  decoration: BoxDecoration(  
    color: Colors.green,  
    borderRadius: BorderRadius.circular(10),  
  ),  
  child: Text('Stylish Text'),  
);
```



なぜ **Flutter** はウィジェット内部で関数やプロパティを使うのか？

- a. なぜウィジェットで関数やプロパティが使われるのか？
 - Flutter では、ウィジェット内部で関数やプロパティを渡すことができる。
 - UI（見た目）とロジック（動作）を1つの場所で定義できるので便利。
 - 従来の UI フレームワークのように、UI とロジックを分ける必要がない。
- b. 例：onPressed でアクションを追加する（ボタンの例）
 - ボタンウィジェットの onPressed に関数を渡すことで、クリック時の動作を指定できる。

- 見た目（UI）と動作（ロジック）を一箇所にまとめることができる。
- 以下はコード例です：

```
ElevatedButton(
  onPressed: () {
    print('Button clicked!'); // This function runs when
the button is pressed
  },
  child: Text('Click Me'), // The text shown inside the
button
);
```

- 説明：
 - ◇ onPressed は関数を渡すためのプロパティ。
 - ◇ ボタンがクリックされると関数が実行される。
 - ◇ UI（テキスト）とロジック（クリック時の動作）が一緒に書かれている。
- c. コードを整理しやすくするために関数を分離する
 - 関数をウィジェットの外に定義し、参照することで、コードが整理され、保守性が向上する。
 - 関数が複雑な場合や、複数の場所で再利用する場合に便利。
 - 例えば、以下は関数を分けて使うコード例です：

```
void buttonClicked() {
  print('Button clicked!'); // This function runs when the
button is pressed
}

ElevatedButton(
  onPressed: buttonClicked, // Just reference the function
by its name
  child: Text('Click Me'), // The text shown inside the
button
);
```

- 関数を分離するメリット：
 - ◇ 再利用性 → 複数のボタンやイベントで関数を再利用できる。
 - ◇ 明確さ → UI コードがスッキリし、デザインに集中できる。
 - ◇ 保守性 → ボタンの動作を変更する場合、関数を 1 箇所変更するだけで済む。
- d. 例：プロパティを使ったスタイリング（Text ウィジェットの TextStyle）
 - プロパティを使うことで、ウィジェットの見た目を簡単にカスタマイズできる。
 - デザイン用のファイルを別に作らなくても、TextStyle などのプロパティでスタイルを定義できる。
 - コードが読みやすく、変更しやすくなる。

- 以下はコード例です：

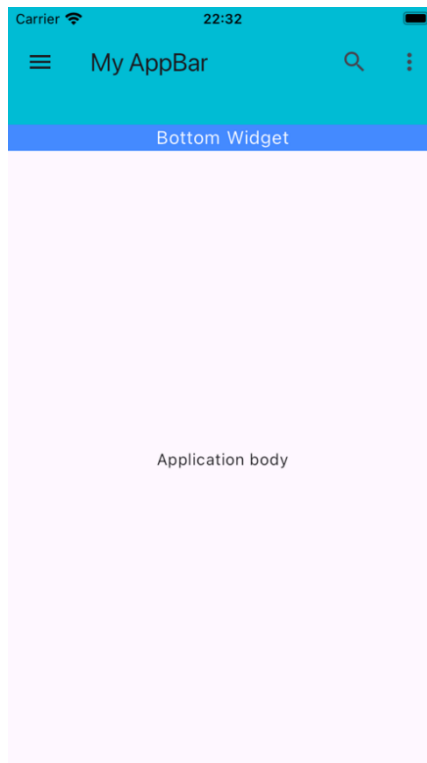
```
Text(  
  'Hello, Flutter!',  
  style: TextStyle( // TextStyle customizes the appearance  
of the text  
    fontSize: 24,    // Makes the text larger  
    fontWeight: FontWeight.bold, // Makes the text bold  
    color: Colors.blue, // Changes the text color to blue  
  ),  
);
```

- 説明:

- ◇ style は TextStyle オブジェクトを受け取るプロパティ.
- ◇ fontSize,fontWeight,color はテキストの見た目を定義するキー.
- ◇ 別のスタイルシートを作らなくても,直接ウィジェット内でカスタマイズできる.

まとめ

- Flutter のキーと値のペアにより,カスタマイズと柔軟な設定が可能になる.
- デフォルト値を上書きすることで,UI の外観や動作を変更できる.
- Flutter の名前付きパラメータは,コードの可読性と保守性を向上させる.
- Flutter のキーと値の構造は JSON に似ているが,目的が異なる.
- ウィジェットのネストによって,UI 要素の構造を明確にできる.
- 関連するコンポーネントをひとまとめにすることで,コードが読みやすくなる.
- スタイル用ウィジェットでラップすることで,カスタマイズが容易になる.
- Flutter の UI はすべてネストされたウィジェットツリーで構築されている.
- ウィジェット内の関数は動作を定義する (例: onPressed) .
- 関数を分離することで,再利用性・明瞭性・保守性が向上する.
- ウィジェット内のプロパティは外観をカスタマイズする (例: Text の style) .
- Flutter は UI とロジックを一箇所に統合しているため,コードの管理がしやすい.



d. AppBar を使うべき場面

- AppBar がよく使われる場面:
 - ◇ アプリの統一感のあるトップバーが必要なとき.
 - ◇ タイトルやナビゲーションボタンを追加したいとき.
 - ◇ 検索,設定,その他のオプションを提供したいとき.
 - ◇ タブや追加コンテンツを AppBar の下に表示したいとき.

Flutter の Text ウィジェット

a. Text ウィジェットとは？

- Flutter アプリでテキストを表示するウィジェット.
- プレーンテキストとスタイル付きテキストの両方をサポート.
- 静的メッセージ,動的更新,レスポンスデザインに対応可能.

b. 主なプロパティ

- 主なプロパティ：

プロパティ	説明	使用例
data	表示するテキスト（必須）.	'Hello, Flutter!'
style	テキストの見た目を変更.	TextStyle(fontSize: 18, color: Colors.blue)

textAlign	テキストの配置(left, center, right).	TextAlign.center
maxLines	最大行数を制限.	maxLines: 2
overflow	テキストのオーバーフロー処理 (ellipsis, fade).	TextOverflow.ellipsis
softWrap	テキストの折り返し設定.	softWrap: true

c. 例：FlutterでのTextスタイルの設定

■ 例のFlutterコード：

```
Text(
  'Welcome to Flutter!', // The text to display
  style: TextStyle(
    fontSize: 20,          // Sets the font size
    color: Colors.green,   // Text color
    fontWeight: FontWeight.bold, // Makes the text bold
  ),
  textAlign: TextAlign.center, // Centers the text
  maxLines: 2,               // Limits to 2 lines
  overflow: TextOverflow.ellipsis, // Adds "..." if the
text is too long
)
```

■ 動作説明:

- ◇ 'Welcome to Flutter!' → 表示するテキスト.
- ◇ style → サイズ,色,太さなどを指定.
- ◇ textAlign: TextAlign.center → テキストを中央揃え.
- ◇ maxLines: 2 → 最大2行までに制限.
- ◇ overflow: TextOverflow.ellipsis → 長すぎる場合“…”を表示.

d. なぜTextウィジェットを使うのか？

■ Textウィジェットを使う理由:

- ◇ 静的・動的テキストの表示が簡単.
- ◇ 整列・折り返し・オーバーフロー処理が可能.
- ◇ どんなレイアウトやUIデザインにも適用できる.

FlutterのContainerウィジェット

a. Containerウィジェットとは？

- スタイル,位置,レイアウトを設定できるカスタマイズ可能な「ボックス」.
- 他のウィジェットをラップし,間隔,色,装飾を適用するのによく使われる.
- Flutterで最も柔軟で一般的に使用されるウィジェットの1つ.